

Automated Strategy Discovery in Crypto Perpetuals via Dynamic Factor Ensembles and Auditable Evaluation

Kaihong Deng

*School of Modern Information Industry, Artificial Intelligence Program, Guangzhou
College of Commerce, Guangzhou, China*

Abstract

Offline backtests are a central evaluation tool for data-driven decision policies, yet their apparent performance can collapse when evaluation mismatches deployment semantics (delays, costs, cash-flow accounting) or when repeated search reuses the same windows. We study automated strategy discovery in cryptocurrency perpetual futures, where fees, slippage, and funding cash flows can dominate profit and loss (PnL) for high-turnover policies. We present an end-to-end protocol that (i) maintains a dynamic, compact factor pool to control feature combinatorics, (ii) searches a modular policy family under STRICT T+1 execution with explicit cost and funding accounting, and (iii) selects configurations via robustness screening (cost stress) and walk-forward evaluation while exporting deterministic, machine-readable artifacts and invariant checks for auditability. From a knowledge-based systems perspective, the protocol represents evaluation semantics and selection rules as explicit knowledge objects, enabling traceable decision support and review. Across four liquid perpetual contracts (BTC/USDT, ETH/USDT, SOL/USDT, AVAX/USDT), we show that fee-only or zero-cost assumptions materially inflate apparent performance relative to a fully costed configuration. Under identical STRICT semantics, two-stage robustness screening tends to reduce drawdowns. Our contribution is validation infrastructure rather than a claim of persistent alpha: the protocol makes offline-to-deployment gaps and selection bias easier to detect, reproduce, and review.

Keywords: crypto perpetual futures, backtesting, evaluation semantics,

Email address: 202312100202@xs.gcc.edu.cn (Kaihong Deng)

1. Introduction

Automated strategy discovery and configuration search are increasingly common in applied data science, especially when decisions are evaluated offline using replay or simulation. However, offline evaluation can be systematically biased when it mismatches deployment semantics and frictions, or when selection repeatedly reuses the same evaluation windows. This failure mode is well known in the backtest-overfitting and data-snooping literature [1–4] and appears in many high-stakes sequential decision settings.

Cryptocurrency perpetual futures (no-expiry derivatives whose prices are anchored to spot via periodic funding payments) provide a concrete case study where these gaps are amplified. Perpetuals introduce a funding cash-flow channel that transfers value between longs and shorts; misaligning funding timestamps with position holding periods can flip realized PnL [5]. Moreover, high turnover makes fee schedules and slippage proxies first-order, so “fee-only” or “zero-cost” backtests can significantly overstate performance [6].

We present a practical protocol for *auditable* strategy discovery in crypto perpetuals. Rather than proposing a new predictor, the core contribution is an evaluation-and-selection procedure that aligns semantics across the entire pipeline and emits deterministic artifacts suitable for review, regression testing, and monitoring. The protocol searches a modular policy family (momentum + mean-reversion with risk overlays and optional factor gating) under strict T+1 execution semantics, and then applies robustness screening and walk-forward evaluation to select configurations that degrade less under cost and time shifts. From a knowledge-based systems perspective, the evaluator and selection logic are represented as explicit, checkable knowledge objects (invariants, gates, and artifact requirements) that support traceable decision support for selecting which configurations should be trusted under realistic constraints [7, 8].

Contributions.

- **Dynamic factor ensembles:** We maintain a compact, refreshed factor whitelist using multi-window IC screening, binarized-sign stability, correlation deduplication, and a diversity constraint to control feature combinatorics.
- **Execution-realistic evaluation:** We enforce STRICT T+1 execution (no lookahead), explicit fees, slippage proxy, and funding accounting, and position caps consistently across tuning, stress tests, and validation.
- **Auditable selection:** We separate *search* (Stage I) from *selection* (Stage II) via robustness gates (cost stress + walk-forward) and export machine-readable artifacts and invariant checks to make decisions reproducible.
- **Case study:** On four liquid perpetual contracts, we quantify how optimistic assumptions inflate performance. We also study how two-stage robustness screening affects returns and drawdowns under identical strict semantics.

2. Problem Setting and Design Goals

We consider automated configuration selection for a pre-specified family of trading policies on cryptocurrency perpetual futures. The goal is not to claim a new alpha model, but to build a selection protocol that makes offline evaluation closer to deployment reality and makes the final selection decision traceable.

2.1. Threat Model: Offline-to-Deployment Gaps

In practice, a strategy can look strong in backtests but fail after deployment due to: (i) **semantic mismatch** (executing on the same bar as signal computation; misaligned funding), (ii) **unmodeled frictions** (fees and slippage for high-turnover policies), and (iii) **selection bias under repeated search** (many trials evaluated on the same windows, increasing the chance of a spurious winner) [1, 9]. Our protocol targets these gaps by enforcing strict execution semantics, explicitly accounting for major cash flows, and gating selection using robustness criteria that must hold out of the initial training objective.

STRICT T+1 timing (schematic).

Bar t close	Compute signals using information up to the close of bar t .
Between t and $t+1$	No same-bar execution; exposure decided at t is not applied within bar t .
Bar $t+1$	Apply the executed exposure e_{t+1} for PnL and cost accounting; turnover costs are charged on executed exposure changes.
Funding	Align funding rates to the bar grid and apply funding cash flows only when the executed exposure spans the funding interval.

Figure 1: A timeline view of STRICT T+1 semantics and funding alignment. The protocol enforces this timing consistently across tuning, stress tests, and walk-forward evaluation.

2.2. STRICT Execution and Cash-Flow Accounting

We evaluate policies under **STRICT T+1 execution**: signals are computed at bar close and positions are applied starting from the next bar, preventing lookahead. Trading frictions are modeled with a reproducible cost profile:

- **Fees** are proportional to executed turnover (absolute position change).
- **Slippage proxy** is modeled as an additional per-trade cost in basis points (bps), also proportional to turnover.
- **Funding** is treated as a signed cash flow (a periodic payment between longs and shorts that anchors perpetual prices to spot) proportional to notional exposure and bar duration, aligned from an 8h funding series to the bar grid.

This profile is not a capacity model and does not capture market impact; instead, it prevents accidental “free lunch” assumptions and enables apples-to-apples comparisons across ablations.

Figure 1 summarizes the timing assumptions that the evaluator enforces; later stages reuse the same semantics rather than relaxing them for tuning convenience.

Sanity check (semantics). To illustrate why semantic alignment matters, we report an example STRICT vs. NAIVE (same-bar, T0) comparison in Table 1. Even when a single-window difference appears modest, enforcing STRICT semantics consistently is essential because automated search can exploit small

Window	Semantics	Ann. Return	Monthly geom	Sharpe	MaxDD	Trades
2022–2023	STRICT (T+1)	-0.221	-0.0206	-2.413	0.221	30
2022–2023	NAIVE (T0)	-0.264	-0.0252	-2.998	0.264	28

Table 1: **Semantic mismatch sanity check (example window)**. NAIVE (T0) executes on the same bar as signal computation and can materially deviate from STRICT (T+1). Because repeated search can exploit even small semantic mismatches, we enforce STRICT semantics consistently across all stages.

Window	Δ Sharpe	Δ m	Δ MaxDD	Δ Trades
2022	-4.172	-0.0820	0.188	0
2023	0.000	0.0000	0.000	0

Table 2: **Semantic mismatch sweep (NAIVE minus STRICT)**. We rerun the same minimal-harness signal model on multiple fixed windows; Δ metrics report NAIVE (T0) minus STRICT (T+1) under the same cost profile. Exact zeros can occur when both semantics induce identical executed exposures (e.g., no-trade windows under tight constraints).

mismatches (in either direction) when repeated across many trials. To reduce the risk of over-interpreting a single window, Table 2 reports a small sweep over multiple fixed yearly windows, summarizing the direction and magnitude of NAIVE-vs-STRICT deviations under the same minimal harness.

2.3. Design Goals and Protocol Invariants

We treat semantic alignment and auditability as first-class design goals. The protocol maintains a small set of invariants designed to catch high-impact bugs (lookahead leakage, funding misalignment, inconsistent cost application) and to ensure deterministic replay. Table 3 summarizes the invariants.

3. Method: Dynamic Factors + Search + Auditable Selection

Figure 2 overviews the protocol. Stage I performs automated search on a fixed training window under STRICT execution semantics. Stage II selects configurations using (a) robustness stress tests under cost perturbations and (b) walk-forward evaluation (WFE) on time-shifted windows, while exporting

ID	Invariant / check	Rationale (what failure it prevents)
INV_TPLUS1	T+1 execution (no lookahead)	Prevents leaking information from the current bar into the executed position.
INV_FUNDING_ALIGN	Funding timestamp alignment	Prevents attributing funding cash flows to the wrong holding period.
INV_COST_LEDGER	Turnover-based costs applied on executed trades	Prevents “free” rebalancing or inconsistent cost application across stages/ablations.
INV_CAPS_RESIZE	Position caps and resizing, not silent rejection	Prevents hidden constraint violations that bias comparisons across trials.
INV_DETERMINISTIC_EXPORT	Deterministic trial logging and exports	Enables audit, regression tests, and exact reproduction of the selection decision.

Table 3: **Protocol invariants.** Invariant IDs make checks reviewable and traceable across stages.

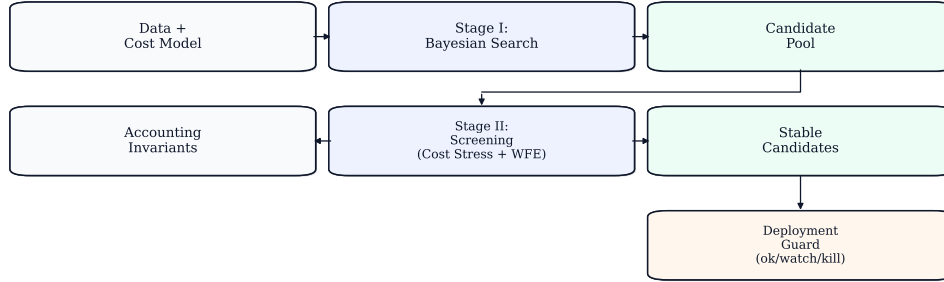


Figure 2: **Protocol overview.** Stage I searches a modular policy family under STRICT T+1 execution and explicit costs. Stage II selects via robustness stress and walk-forward evaluation, producing auditable artifacts and lightweight guardrails.

deterministic artifacts and invariant checks for auditability. Search can be aggressive; selection must be evidence-based and auditable.

3.1. Policy Family: Signals, Factors, and Risk Overlays

We model exposure $e_t \in [-e_{\max}, e_{\max}]$ as a bounded scalar target updated at bar close and executed from the next bar (T+1). The core score combines a momentum sub-score (exponential moving average (EMA)-based trend) and a mean-reversion sub-score (Bollinger-style deviation), then applies optional modules and risk overlays:

$$s_t = g_{\text{combine}}(s_t^{\text{mom}}, s_t^{\text{mr}}, s_t^{\text{opt}}; \theta), \quad (1)$$

where s_t^{opt} may include breakout/volatility-momentum/carry features, and θ includes both numerical hyperparameters and discrete module choices.

Exposure is optionally smoothed, quantized, and subject to constraints (e.g., minimum holding time, cooldown) to suppress churn (excessive trading due to frequent position adjustments). We also allow a factor channel that gates or tilts exposure based on a small subset of factors selected from a curated pool.

Executed PnL and cash flows.. Let p_t be the close price of bar t and let e_t be the executed exposure during bar t (computed from signals at $t - 1$ under STRICT T+1). We treat the evaluator as a cash-flow ledger that decomposes net returns into raw returns and explicit costs. At a high level, per-bar net return can be written as:

$$r_t^{\text{net}} = e_t \cdot r_t^{\text{raw}} - c_t^{\text{fee}} - c_t^{\text{slip}} + \phi_t^{\text{fund}}, \quad (2)$$

where r_t^{raw} is the asset return over bar t , c_t^{fee} and c_t^{slip} are turnover-proportional costs, and ϕ_t^{fund} is the signed funding cash flow aligned to the bar grid. The key requirement is consistency: the same ledger logic is used in Stage I tuning, Stage II stress, and walk-forward evaluation.

3.2. Dynamic Factor Pool Maintenance

Feature combinatorics are a major source of overfitting in automated search. We therefore maintain a compact whitelist of factors. Given a candidate set of implemented factors, we compute forward returns and rank-based information coefficient (IC) on multiple lookback windows, track both raw IC and binarized-sign IC (factor > 0) for stability, then: (i) hard-drop invalid or near-constant factors, (ii) greedily deduplicate by correlation on binarized sequences, and (iii) enforce a diversity constraint against a fixed “anchor” proxy to avoid collapse into a single style.

IC and binarized-sign stability.. For a factor series f_t and a forward return $r_{t \rightarrow t+h}$, we compute a rank-based IC (Spearman correlation) within each month (or evaluation window):

$$\text{IC}(f, r) = \text{corr}(\text{rank}(f_t), \text{rank}(r_{t \rightarrow t+h})). \quad (3)$$

To discourage brittle reliance on magnitude, we also compute a binarized-sign variant by thresholding f_t at zero (or a small quantile) and measuring the correlation between the binary indicator and forward returns. We score factors by cross-window stability (e.g., median IC and fraction of windows with consistent sign), which favors factors whose directional association persists across time slices.

Pool	Candidates	Active	$\mathbb{E}[IC_{\text{bin}}]$	Dedup thr
Unfiltered	20	20	0.0156	—
Curated (active)	20	12	0.0180	0.95

Table 4: **Factor pool snapshot.** The curated pool is produced by multi-window IC screening and binarized-correlation deduplication, with a small diversity constraint ($\text{min_diverse}=3$).

Correlation deduplication and diversity. Even after stability screening, many candidate factors are near-duplicates. We therefore deduplicate greedily by correlation on binarized sequences: sort factors by stability score, keep the top factor, and drop any remaining factor whose binarized sequence is highly correlated with a kept factor. Finally, we enforce a lightweight diversity constraint relative to an “anchor” proxy to avoid the active pool collapsing into a single style that automated search can overfit.

Factor pool refresh (simplified).

1. Input an open-high-low-close-volume (OHLCV) stream, a list of months $\mathcal{M}$, and a candidate factor set $\mathcal{F}$.
2. For each $m \in \mathcal{M}$, compute forward returns and each factor series, then compute rank-IC and a binarized-sign IC, marking invalid/near-constant factors.
3. Score factors by stability across windows (median IC, sign-consistency), then apply correlation deduplication on binarized sequences.
4. Enforce a diversity constraint against an anchor proxy and output a compact active factor whitelist.

Table 4 reports an example summary of the maintained factor pool.

3.3. Stage I: Black-Box Search Under STRICT Semantics

Stage I runs black-box optimization over a mixed discrete–continuous space under STRICT evaluation. Each trial uses the same evaluator (STRICT T+1 + cost ledger + funding alignment) as later stages. Invalid configurations are treated as first-class outcomes: we record invalidity reasons (e.g., near-constant factor inputs, degenerate parameter combinations, no-trade outcomes under constraints) and assign a low sentinel score. This prevents the optimizer from hiding constraint violations and makes search pressure measurable.

Gate ID	Stage	Rule (reviewable / numeric)
GATE_COST_STRESS	II-A	Stress grid: taker fee (bps) $\in \{3, 4, 6\}$ and funding multiplier $\in \{0.5, 1.0, 1.5\}$ (slippage multiplier fixed at 1.0 in this snapshot); compute m_{nom} at (4 bps, funding multiplier 1.0) and m_{worst} as the minimum m over the grid. Gate: require $m_{worst} \geq 0$ and $\Delta = m_{nom} - m_{worst} \leq 0.02$ to avoid candidates whose performance relies on optimistic costs or favorable funding scaling.
GATE_WFE	II-B	Walk-forward windows: win_bars= 2000, step_bars= 500 (4h; 14 windows in the BTC snapshot). Gate: require worst-window proxy $\min m \geq 0.04$, mean MaxDD ≤ 0.15 , and mean cost/bar ≤ 3.0 bps to favor time-shift stability and low-churn configurations.

Table 5: **Selection gates as explicit rules.** Gate IDs make selection rules reviewable and traceable across stages.

3.4. Stage II: Robustness-Gated Selection (Stress + WFE)

Stage II applies two robustness gates: (a) **cost robustness stress** re-evaluates shortlisted candidates under perturbed cost parameters to quantify degradation and churn, and (b) **walk-forward evaluation** measures time-shift degradation across overlapping windows without refitting. Final ranking is stability-aware, combining window-level performance summaries (mean/min monthly geometric return proxy), drawdowns, and churn proxies (switch density, cost/bar).

Validity flags and sentinel scoring. Automated search over a mixed discrete-continuous space frequently produces invalid configurations. We explicitly log invalidity reasons and assign a low sentinel score. This prevents the optimizer from “hiding” constraint violations and makes search pressure measurable.

Complexity and determinism. The protocol is designed to be deterministic given a fixed data slice, cost profile, and random seed. For each trial, the dominant cost is one backtest pass plus feature computation for the chosen factor subset. The dynamic factor pool refresh is performed on a fixed schedule and outputs a compact whitelist, reducing the discrete branching factor seen by Stage I.

Auditable selection (high-level). Algorithm 1 summarizes the two-stage selection logic and its exported evidence record.

Figure 3 summarizes the selection logic at a high level; the experiments report both intermediate robustness summaries (stress and WFE) and final validation outcomes under the same STRICT semantics.

Algorithm 1: Two-stage robustness-gated selection (high-level).

Input: training window $\mathcal{W}_{\text{tr}}$, walk-forward windows $\{\mathcal{W}_k\}$, cost profile $\mathcal{C}$, search budget N .

Output: selected configuration $\hat{\theta}$ and an auditable run record.

1. **Stage I (search):** sample θ_i for $i = 1, \dots, N$ via TPE (Tree-structured Parzen Estimator) or random search; evaluate under STRICT T+1 and $\mathcal{C}$ on $\mathcal{W}_{\text{tr}}$; log metrics and validity flags; assign sentinel score to invalid trials.
2. **Shortlist:** keep top- K valid trials by the Stage I objective plus a small diversity buffer.
3. **Stage II-A (cost stress):** reevaluate each shortlisted θ under a small grid of perturbed frictions (taker fee and funding scaling in this paper snapshot); compute degradation and churn proxies; drop configurations that fail the explicit stress gate in Table 5.
4. **Stage II-B (walk-forward):** evaluate remaining θ on $\{\mathcal{W}_k\}$ without refitting; summarize mean/min window-level performance, drawdown, and cost/churn; drop configurations that fail the explicit WFE gate in Table 5.
5. **Select and export:** choose $\hat{\theta}$ by a stability-aware ranking and export deterministic artifacts (trial logs, ledgers, summaries, invariant reports) for audit.

Figure 3: The selection protocol makes window reuse, cost sensitivity, and time-shift degradation explicit, and keeps invalid configurations as first-class outcomes.

Window-level monthly geometric return proxy. Several tables summarize window-level performance using a monthly geometric return proxy m . Given per-bar net returns r_t^{net} and a set of bars $\mathcal{T}$ within a window, we define:

$$m(\mathcal{T}) = \left(\prod_{t \in \mathcal{T}} (1 + r_t^{\text{net}}) \right)^{\frac{B_{\text{month}}}{|\mathcal{T}|}} - 1, \quad (4)$$

where B_{month} is the number of bars per month for the evaluation frequency. This yields a scale-consistent proxy for comparing windows of different lengths while remaining grounded in the same net-return ledger.

3.5. Auditability: Deterministic Artifacts and Invariants

Each stage exports deterministic, machine-readable artifacts that make selection traceable. A reviewer can inspect which trials were evaluated and

Artifact (machine-readable)	Purpose	Typical fields
Trial table (Stage I)	Reproduce selection inputs	params, objective, validity flags
Window metrics	Diagnose stability	m (monthly return proxy), Sharpe, MaxDD, switches
Cost ledger	Reconcile cash flows	fees, slippage proxy, funding, cost/bar
Stress + WFE summaries	Quantify degradation	mean/min window stats, rank deltas
Invariant checks	Detect semantic bugs	T+1, funding alignment, cap enforcement

Table 6: **Audit artifacts exported by the protocol.** All stages emit deterministic, machine-readable outputs to support review, regression testing, and post-deployment monitoring.

Knowledge object	Fields (reviewable / machine-checkable)
Invariant	id, description, check_name, failure_rationale
Gate	id, stage, input_metrics, rule_logic, action (pass/penalize/reject)
Artifact requirement	type, schema_fields, provenance (stage/trial/window), path
Run record	seed, windows, cost profile, trial table, summaries, gate outcomes
Explanation	failed_checks, violated_invariants, rank_deltas, human-readable rationale

Table 7: **Protocol as a knowledge inventory.** The contribution is not only an evaluator, but an explicit, reviewable knowledge model of what must hold (invariants), how decisions are filtered (gates), what evidence must be produced (artifacts), and how failures are explained.

which constraints invalidated candidates. Reviewers can also inspect cost decomposition and rank changes under stress tests and walk-forward windows. Table 6 summarizes the key artifact types.

3.6. Protocol Knowledge Model: What is Represented and What is Checkable

To align with a knowledge-based systems perspective, we treat the protocol specification itself as a compact knowledge model in the classical sense of explicit, checkable knowledge objects and rule-like filters [7, 8]. It explicitly represents (i) *what must hold* (invariants), (ii) *how selection is filtered* (gates), (iii) *what evidence must be exported* (artifact requirements), and (iv) *how outcomes are explained* (failure rationales and rank deltas). Table 7 summarizes the core objects and their reviewable fields.

The IDs in Tables 3 and 5 provide a compact, reviewable handle on these knowledge objects in the manuscript. Table 8 makes the representation explicit as a small executable schema: rules (invariants/gates) applied to structured evidence records.

Table 9 illustrates the kinds of lightweight queries this representation enables for audit and decision support.

Policy / schema item	Definition (fields → checkable effect)
Invariant spec	{ <code>id</code> , <code>description</code> , <code>check_name</code> , <code>failure_rationale</code> } → a deterministic pass/fail check applied to each trial/stage; failures invalidate results and are logged.
Gate spec	{ <code>id</code> , <code>stage</code> , <code>metric</code> , <code>op</code> , <code>value</code> , <code>rationale</code> } → a rule evaluated on aggregated evidence fields (e.g., <code>min_m</code> , <code>mean_maxdd</code> , <code>cost_bar_bps</code>); see Table 5.
Evidence record	{ <code>trial_id</code> , <code>freq</code> , <code>windowing</code> , <code>mean_m</code> , <code>min_m</code> , <code>mean_maxdd</code> , <code>cost_bar_bps</code> , (optional) <code>stress summary</code> , (optional) <code>perturb quantiles</code> } → a machine-checkable summary sufficient to reproduce Stage-II gate decisions without narrative.
Aggregation rule	“per-trial median over repeated audits” → reduces sensitivity to a single run instance when multiple audit summaries exist; used in Table 15.
Explanation object	{ <code>gate_outcomes</code> , <code>violated_invariants</code> , <code>rank_deltas</code> , <code>human_rationale</code> } → supports reviewer queries: completeness/consistency/explanation (Table 9).

Table 8: **Executable policy schema (paper-facing)**. The protocol is represented as explicit knowledge objects (invariants and gate rules) applied to structured evidence records. This makes selection decisions checkable without relying solely on narrative.

Query type	Question (template)	Returns (reviewable fields)
Completeness	Do all reported result tables have the required artifacts (trial table, cost ledger, WFE/stress summaries) for the stated window/cost profile?	Artifact types, schema fields, provenance (stage/trial/window), missing items
Consistency	Did any invariant fail during tuning/stress/WFE for the configuration(s) underlying a reported table?	Failed checks (by invariant ID), failure rationale, affected stages
Explanation	Why was a high-performing Stage I trial rejected or penalized in Stage II?	Gate outcomes, rank deltas under stress/WFE, violated invariants, churn/cost diagnostics

Table 9: **Example reviewer queries enabled by the knowledge inventory**. Invariants, gates, and artifact requirements make the selection decision checkable without relying on narrative alone.

Reviewer queries enabled by the model.. The explicit representation supports lightweight, rule-like queries that are useful for audit and decision support, e.g.: (i) *completeness*: are all required artifacts present for the reported tables?; (ii) *consistency*: did any invariant fail during tuning/stress/WFE (thus invalidating a claimed result)?; and (iii) *explanation*: why was a high-performing Stage I trial rejected or penalized in Stage II (churn, cost sensitivity, or worst-window degradation)?

Knowledge-based perspective (KBS fit).. Although the trading policy family is numerical, the *evaluation protocol* is explicitly represented as auditable knowledge: protocol invariants define what semantics must hold, and artifact inventories define what evidence must be exported for review. This represen-

tation makes evaluation decisions explainable and reproducible in the same spirit as knowledge-based decision support.

4. Experiments

We evaluate the protocol on four liquid perpetual futures contracts (BTC/USDT, ETH/USDT, SOL/USDT, AVAX/USDT). All reported results use the same STRICT T+1 semantics and explicit cost accounting. Comparisons differ only in the selection protocol or controlled ablations.

4.1. Experimental Setup and Metrics

We treat this paper as an applied evaluation study: the modeling family is intentionally modular and simple, while the evaluation semantics and selection protocol are designed to be strict and auditable. We report window-level summaries to emphasize time-shift robustness rather than a single best-case backtest. Unless stated otherwise, returns are reported in decimal units, MaxDD is in $[0, 1]$, and costs are aggregated from the same per-bar ledger used throughout the pipeline. Stage I uses a best-of- N objective on a fixed training window; we report optimizer behavior across budgets N in Table 11. Stage II operates on a shortlisted candidate set and re-ranks candidates based on robustness stress and walk-forward summaries, rather than on the Stage I objective alone. We emphasize that many configurations are invalid under realistic constraints; we log invalidity and use sentinel scoring so that budgets are comparable. For reviewer convenience, Table 10 provides a compact claim-to-evidence map, and Table 6 summarizes exported artifacts; together these support quick checks of what was evaluated, what was filtered by gates, and where each claim is supported.

Cost profile (nominal).. Unless stated otherwise, we use a fixed, reproducible cost profile: initial equity 10,000, maximum leverage 5, a notional cap of 50,000, taker fee 4 bps per unit turnover, and a linear slippage proxy of 2 bps per unit turnover (see `config/cost_profile.json`). Funding cash flows use an aligned 8h funding-rate series resampled to the bar grid.

4.2. Data and Preprocessing

We use OHLCV bars with an aligned funding-rate series (8h funding resampled/forward-filled to the bar grid). Unless stated otherwise, we evaluate on 1h bars; some robustness and selection snapshots use 4h bars for stability,

Claim	What is claimed (scope-limited)	Evidence in this manuscript
C1	STRICT T+1 semantics can differ from same-bar (NAIVE) evaluation, and repeated search can exploit small mismatches; therefore the evaluator enforces STRICT semantics consistently.	Table 1, Table 2, Fig. 1
C2	Ignoring major cash flows (fees, slippage proxy, funding) can materially inflate apparent performance in high-turnover strategies.	Table 12
C3	Robustness-gated selection (cost stress + walk-forward) can reduce drawdowns and reject fragile, high-churn winners under identical STRICT semantics.	Table 13, Table 14, Table 16
C4	Best-of- N search on a fixed window exhibits substantial selection bias; a lightweight CSCV/PBO diagnostic makes this risk visible.	Table 17
C5	The protocol is auditable because it exports deterministic artifacts and enforces explicit invariants and gates, enabling traceable decision support.	Table 6, Table 3, Table 5, Table 7

Table 10: **Claim-to-evidence index.** We keep claims intentionally scope-limited (validation infrastructure rather than persistent alpha) and point each claim to the specific evidence object(s) in the manuscript.

and each table reports its frequency explicitly when relevant. Window labels in tables indicate the date span (e.g., 2022–2023). To reduce unintentional leakage, the evaluator uses only information available up to the current bar close to compute signals, and executes positions starting from the next bar. For funding, we align cash flows to the bar grid and apply them only when exposure spans the funding interval, so that funding timing is consistent with STRICT T+1 holding periods.

Windowing protocol. To avoid reporting a single fragile backtest, we use multiple time windows. Stage I uses a fixed training window; Stage II-A stress is run on train+validation; Stage II-B walk-forward evaluation runs on a sequence of windows without refitting. For WFE, we choose overlapping windows with the same configuration and summarize both mean behavior and worst-window behavior. This makes time-shift degradation explicit and reduces the chance that selection is driven by a single lucky segment.

Cost profile and stress grid. The baseline cost profile includes turnover-proportional fees and a linear slippage proxy, plus funding cash flows aligned to the bar grid. Stage II-A stress perturbs these cost parameters (e.g., increasing fee and slippage bps) to quantify how quickly a candidate degrades and whether it relies on excessive churn. We report both performance and

churn proxies (switch density, cost/bar) to avoid selecting configurations that are profitable only under unrealistically low friction.

4.3. Stage I: Search Baselines

Stage I is a best-of- N search on a fixed training window. We compare Bayesian optimization (Tree-structured Parzen Estimator; TPE) [10] against random search [11] under identical STRICT evaluation. Table 11 reports an example comparison: as the budget increases, TPE achieves higher best-of- N scores, while random search often yields invalid trials under constraints (scored with a sentinel).

Table 11: **Optimizer baseline under identical STRICT training window.** Best-of- N objective (annual). Random values are mean $\pm$ std; repeats reflect best-of- N plateaus; invalid trials score -1.0 .

Budget N	TPE best	Random best (mean $\pm$ sd)
10	0.0174	-1.0000 $\pm$ 0.0000
20	0.0362	-0.7832 $\pm$ 0.4848
40	0.1172	-0.7832 $\pm$ 0.4848
60	0.1334	-0.7832 $\pm$ 0.4848
80	0.1508	-0.6638 $\pm$ 0.4915
100	0.1590	-0.4600 $\pm$ 0.5163
120	0.1590	-0.3580 $\pm$ 0.6763

4.4. Costs and Cash-Flow Accounting Ablation

Crypto perpetuals highlight how optimistic assumptions can dominate conclusions. Table 12 decomposes cumulative fees, slippage proxy, and funding cash flows for an example setting; fee-only or zero-cost assumptions materially inflate apparent performance compared to a fully costed backtest.

4.5. Stage II-A: Cost Robustness Stress

Robustness stress evaluates shortlisted candidates under perturbed cost parameters to quantify degradation and churn. In the 4h snapshot, the stress grid varies taker fees (3/4/6 bps) and funding scaling (0.5/1.0/1.5 $\times$) around the nominal cost profile (4 bps, 1.0 $\times$ funding); the slippage multiplier is fixed at 1.0 in this snapshot. Table 13 summarizes how the monthly proxy m degrades under the worst setting for each candidate and reports a churn proxy (switch density).

Variant	Ann. Return	Sharpe	MaxDD	Trades	Fees	Slip	Funding	Total
Rigorous (fees+slip+fund)	2.726	2.049	0.265	636	0.371	0.742	0.080	1.192
Fee-only (no slip/fund)	4.308	2.506	0.262	636	0.371	0.000	0.000	0.371
Zero-cost (caps fixed)	5.225	2.711	0.260	535	0.000	0.000	0.000	0.000
Zero-cost (caps relaxed)	5.225	2.711	0.260	535	0.000	0.000	0.000	0.000

Table 12: **Accounting ablation (BTC, example trial/window)**. Costs are decomposed into cumulative fees, slippage proxy, and funding cash flows (same units as equity); ignoring costs can materially inflate apparent performance.

Trial	m_{nom}	m_{worst}	Δ	$MaxDD_{nom}$	$MaxDD_{worst}$	SD_{nom}	SD_{worst}
15	0.015	0.014	0.001	0.348	0.349	0.186	0.186
16	-0.006	-0.007	0.001	0.473	0.477	0.186	0.186
18	0.024	0.023	0.001	0.359	0.362	0.178	0.178
21	0.017	0.016	0.001	0.203	0.208	0.188	0.188
23	0.000	-0.001	0.001	0.323	0.326	0.130	0.130

Table 13: **Cost robustness grid stress (BTC/USDT, 4h)**. We vary taker fees (3/4/6 bps) and funding scaling (0.5/1.0/1.5 $\times$) around the nominal setting (4 bps, funding scaling 1.0 $\times$) and report worst-case degradation over the grid. SD is a churn proxy (switch density = position switches per bar).

4.6. Stage II-B: Walk-Forward Evaluation (WFE)

WFE quantifies time-shift degradation by evaluating a fixed configuration over overlapping windows without refitting. We use a deterministic windowing scheme with `win_bars`= 2000 and `step_bars`= 500 (4h; 14 windows in the BTC snapshot), and summarize both mean and worst-window behavior. Table 14 reports BTC results for three candidates over 14 windows, including mean/min window-level monthly return proxy, MaxDD, churn, and cost/bar.

How WFE is used in selection.. We treat the WFE summary as a robustness gate with explicit numeric thresholds (Table 5): candidates are rejected when worst-window performance falls below a positive margin ($\min m < 0.04$), when average drawdown is too high (mean MaxDD > 0.15), or when churn-cost is excessive (cost/bar > 3 bps). This forces the final selection to be stable under both *cost variation* (stress) and *time variation* (WFE). As a batch sanity check beyond any single configuration, Table 15 aggregates 4h audit summaries (271 unique trials) and shows that WFE-accepted candidates have a higher rate of non-negative perturb-holdout p05 (5th percentile) outcomes.

Trial	Mean m	Min m	Mean MaxDD	Switches	Cost/bar (bps)
9966	0.126	0.084	0.125	70.1	1.90
9804	0.123	0.075	0.131	62.0	1.92
8758	0.120	0.067	0.127	68.9	1.86

Table 14: **Walk-forward evaluation (BTC, 4h; win_bars=2000, step_bars=500).** m is a window-level monthly geometric return proxy. We report mean/min m , mean MaxDD, churn (mean position switches), and mean cost/bar as aggregated in the audit summaries for each trial.

Metric	All	WFE-accepted	WFE-rejected
N audit summaries (raw)	366	–	–
N unique trial ids	271	51	220
Accept rate (%)	18.8	–	–
Median WFE min m	0.027	0.050	0.019
Median WFE mean MaxDD	0.068	0.099	0.063
Median WFE cost/bar (bps)	0.564	0.987	0.470
Perturb-holdout m p05 ≥ 0 (%)	42.4	68.6	36.4
Median perturb-holdout m p05	-0.004	0.001	-0.007
Median perturb-holdout m p50	0.044	0.053	0.041
Median perturb-holdout m p95	0.081	0.091	0.076
Median perturb-holdout MaxDD p95	0.198	0.222	0.190

Table 15: **WFE gate batch summary (4h audit summaries).** We aggregate existing audit summaries at the trial level (median across repeated audits) and apply a numeric WFE gate (min $m \geq 0.04$, mean MaxDD ≤ 0.15 , cost/bar ≤ 3.0 bps). All included trials share the same walk-forward windowing (win_bars=2000, step_bars=500). We then compare perturb-holdout robustness quantiles between accepted and rejected sets (p05/p50/p95 denote the 5th/50th/95th percentiles).

4.7. One-Stage vs. Two-Stage Selection

To isolate the effect of robustness screening, we compare one-stage selection (choose the best Stage I trial on the training window) against two-stage selection (Stage II robustness gates before final choice) under identical STRICT semantics. Table 16 reports validation outcomes across assets. Two-stage screening does not guarantee higher returns; its main effect is to reduce drawdowns and to avoid fragile high-churn configurations.

Interpreting extreme outcomes (SOL/AVAX). Table 16 includes extremely large annualized returns for some assets under one-stage selection (e.g., AVAX) as well as dramatic changes after Stage II filtering. These extremes are a

Asset	Sel.	Ann. Return	Sharpe	MaxDD
BTC/USDT	1-stage	0.822	1.238	0.318
BTC/USDT	2-stage	0.572	1.070	0.231
ETH/USDT	1-stage	1.659	2.157	0.186
ETH/USDT	2-stage	0.720	1.368	0.201
SOL/USDT	1-stage	10.650	3.398	0.239
SOL/USDT	2-stage	39.766	4.208	0.242
AVAX/USDT	1-stage	227.088	3.851	0.621
AVAX/USDT	2-stage	1.719	4.185	0.096

Table 16: **One-stage vs. two-stage selection (validation, 1h).** Same STRICT protocol and costs; two-stage adds robustness screening before selection. Validation windows are asset-specific held-out periods (reported in the underlying run summaries). Annualized returns can be extreme in volatile assets; interpret together with worst-window summaries, churn proxies, and MaxDD.

symptom of the threat model rather than a positive claim: best-of- N selection on a fixed window can prefer configurations whose apparent performance depends on a small number of volatile segments, high churn, or cost-sensitive behavior. Annualization can further amplify scale in volatile assets; we therefore interpret annualized returns together with worst-window summaries (min m), churn proxies, and drawdowns rather than as a standalone objective. Stage II filtering can therefore either (i) reduce drawdown by rejecting high-churn candidates, or (ii) materially reduce headline returns by discarding fragile winners. This is why we emphasize worst-window and churn-aware summaries in Stage II (Tables 13 and 14) and report selection-bias diagnostics (Table 17) alongside validation metrics.

4.8. Selection Bias Sanity Check (CSCV/PBO)

Repeated search on a fixed window increases the probability of selecting an overfit configuration [1]. As a lightweight sanity check, we estimate the probability of backtest overfitting (PBO) via combinatorially symmetric cross-validation (CSCV) on a Stage I candidate set. Table 17 reports an example result (Sharpe-based PBO ≈ 0.800), suggesting that many in-sample winners do not preserve rank out-of-sample and motivating robustness-gated selection.

Metric	Segments	Candidates	CSCV splits	PBO
Sharpe	8	40	70	0.800

Table 17: **Selection-bias sanity check via CSCV/PBO (BTC).** We compute Sharpe on the monthly return series under STRICT semantics and split the series into 8 contiguous segments; the candidate pool is a fixed Stage-I snapshot (top-by-score) exported from the same search run. PBO is the estimated probability that the in-sample winner fails to preserve rank out-of-sample under combinatorially symmetric cross-validation.

4.9. Targeted Ablation: Funding Gates

Perpetual funding can be a significant cash-flow channel and can interact with regime filters. Table 18 disables funding-related gating/bias rules while keeping STRICT execution and cost accounting fixed; in this example setting, funding gates have a small effect compared to the dominant cost channels.

Table 18: **Funding gate ablation (example trial).** Same STRICT protocol and costs; only funding gates/bias disabled.

Variant	Train monthly geom	Train MaxDD	Val monthly geom	Val MaxDD
Full	0.148	0.265	0.039	0.231
NoFundingGates	0.144	0.265	0.039	0.231

5. Related Work

This paper sits at the intersection of (i) evaluation validity under repeated search, (ii) execution- and cost-aware backtesting for trading policies, and (iii) black-box optimization for configuration search. We briefly position the protocol relative to these strands.

5.1. Selection Bias Under Repeated Backtests

When many configurations are evaluated on the same windows, selecting the best-looking backtest can induce large optimism even if each individual test is “strict”. This problem is studied in the data-snooping and multiple-testing literature for trading rules [2–4], and more broadly in backtest overfitting diagnostics [1]. Practical mitigations include separating *search* from *selection*, using stability criteria on held-out or time-shifted windows, and reporting

diagnostics that make window reuse and selection pressure visible [9]. Related practitioner-facing discussions emphasize that selection pressure and repeated tuning can dominate backtest conclusions even with seemingly reasonable pipelines [12]. Our protocol operationalizes these ideas by treating best-of- N search as Stage I and requiring candidates to pass out-of-objective robustness gates (cost stress and walk-forward evaluation) in Stage II.

5.2. Execution Semantics, Costs, and Cash Flows

Backtest validity depends on aligning evaluation semantics with deployment constraints. Execution delays (e.g., signal-at-close, execute-next-bar) and the accounting of trading costs are first-order for high-turnover strategies, where ignoring frictions can dominate conclusions [6]. Perpetual futures further add a funding cash-flow channel whose timing and sign depend on position holding periods and the contract mechanics [5]. While our cost model is intentionally simple (fees + linear slippage proxy + funding cash flows), the key contribution is consistent application: STRICT T+1 execution, cost accounting, and funding alignment are enforced identically during tuning, stress tests, and walk-forward evaluation. This emphasis mirrors a broader theme in applied ML systems: small semantic mismatches can be systematically exploited by automated search, so evaluation must be treated as part of the method rather than as an afterthought. Recent empirical work studies risk and factor structure in cryptocurrency markets [13, 14], and pricing and microstructure of perpetual futures [15]. These studies reinforce that crypto markets are highly time-varying; our goal is therefore not to declare a stable alpha, but to provide a protocol that makes time variation, costs, and cash flows explicit during automated search.

5.3. Automated Configuration Search

Stage I uses black-box optimization to explore a mixed discrete–continuous configuration space, following standard practice in hyperparameter optimization [10, 11, 16]. In this setting, invalid configurations (degenerate parameters, no-trade outcomes under constraints, unstable factor inputs) are common and must be treated as first-class outcomes rather than silently filtered. Our protocol logs validity flags and assigns sentinel scores so that optimization pressure does not hide constraint violations, and so that selection decisions remain auditable. More generally, the combination of strict evaluators and auditable trial logs can be viewed as a lightweight governance layer for applied AutoML in domains where the cost of optimistic evaluation is high. This

aligns with the broader AutoML perspective that optimization algorithms can be effective only when the objective and constraints are well-specified and reproducible [17, 18].

6. Discussion and Limitations

Our results reinforce a pragmatic point for applied data science: evaluation and selection infrastructure can be as important as modeling. In domains with frictions and cash-flow constraints, small semantic mismatches can be amplified by repeated search and lead to brittle “winners” that do not survive deployment.

What the protocol is (and is not).. The protocol is a validation-and-selection system for a pre-specified policy family. It does not prove persistent alpha and does not remove the need for careful out-of-sample testing; rather, it makes performance claims harder to inflate by enforcing consistent semantics and by exporting artifacts for review. The CSCV/PBO diagnostic [1] suggests substantial residual selection bias even after robustness gating, aligning with the broader warning that best-of- N backtests can be deceptive [9].

Capacity and market impact.. Our cost model includes fees, a linear slippage proxy, and funding cash flows, but does not model market impact, queue position, or capacity constraints. This is a deliberate scope choice for an applied study: the goal is to ensure semantic alignment and reproducibility; in production settings, impact-aware execution models or conservative capacity bounds are required.

Invalid configurations as first-class outcomes.. Automated search over mixed discrete–continuous spaces can generate invalid configurations. Examples include:

- near-constant factors,
- degenerate parameter combinations,
- no-trade outcomes under caps and cooldowns.

Treating these as first-class outcomes—logging them explicitly and assigning a sentinel score—improves auditability and makes it possible to quantify constraint tightness and search-space pathologies.

Broader applicability.. While crypto perpetuals are a deliberately harsh test bed, the same protocol pattern applies wherever sequential decision policies are selected based on offline evaluation under frictions (e.g., supply chain optimization or healthcare resource allocation): enforce semantics, account for costs, separate search from selection, and emit artifacts suitable for review, regression testing, and explicit knowledge-driven decision support.

Assurance-layer checks (out of scope).. The protocol emphasizes *evaluation* and *selection* semantics. Complementary assurance mechanisms such as pre-run consistency checks, runtime monitoring, and rule-based kill-switches can further reduce deployment risk, but are orthogonal to the contribution in this paper and are not required to reproduce the reported experiments.

KBS framing.. From a Knowledge-Based Systems perspective, the key contribution is the explicit representation of evaluation semantics and selection logic as a reviewable knowledge inventory: invariants, gates, and artifact requirements. This enables traceable decision support for “which configuration should be trusted” under realistic constraints, in the spirit of explicit knowledge representation and rule-based filtering [7, 8].

Threats to validity.. First, the cost model (fees and a linear slippage proxy) does not capture market impact, queue position, or capacity constraints; a configuration that is robust under our stress grid can still fail when scaled. Second, our factor pool maintenance is a pragmatic heuristic; while it controls combinatorics, it does not provide a formal guarantee against overfitting. Third, our analysis is limited to a small set of liquid perpetual contracts; the protocol is intended to be reusable, but the empirical conclusions may vary by venue, fee schedule, and market microstructure.

What we claim.. We do not claim persistent alpha. We claim that, in this domain, strict execution semantics, explicit cash-flow accounting, and robustness-gated selection materially reduce common offline-to-deployment evaluation gaps and make automated search outputs easier to audit.

7. Conclusion

We presented an auditable protocol for automated strategy discovery in cryptocurrency perpetual futures. The protocol maintains a compact

dynamic factor pool. It evaluates candidates under STRICT T+1 execution with explicit fees, slippage proxy, and funding accounting. It selects configurations via robustness stress tests and walk-forward evaluation while exporting deterministic artifacts and invariant checks. Empirically, optimistic cost assumptions materially inflate apparent performance. With strict semantics fixed, two-stage screening tends to reduce drawdowns. The main outcome is a knowledge-based validation infrastructure that helps detect offline-to-deployment gaps and selection bias, improving the credibility and reproducibility of applied strategy search.

Appendix A. Appendix: Reproducibility and Additional Details

This appendix collects implementation-level details that support reproducibility and review. The goal is not to claim universal optimality of any particular cost model or factor recipe, but to make the *evaluation semantics* and *selection logic* explicit and consistent across stages.

Appendix A.1. STRICT T+1 Semantics (Operational Definition)

All experiments adopt STRICT T+1 execution: signals are computed at bar close t using information available up to that close, and the executed exposure applies from bar $t + 1$ onward. This prevents same-bar execution and makes it harder for automated search to exploit subtle lookahead.

Appendix A.2. Cost and Funding Accounting (Ledger View)

The evaluator maintains a per-bar cash-flow ledger with three main components:

- **Fees:** proportional to turnover, i.e., to the absolute change in executed exposure.
- **Slippage proxy:** an additional proportional cost in bps, also applied to turnover.
- **Funding:** a signed cash flow proportional to notional exposure and bar duration, using an aligned funding-rate series.

The important property is stage-consistency: the exact same ledger logic is used for Stage I tuning, Stage II stress tests, and walk-forward evaluation.

Appendix A.3. Dynamic Factor Pool Maintenance (Expanded Description)

To control feature combinatorics, we maintain a compact active whitelist of factors. Given a candidate factor set, we compute multi-window IC summaries (including a binarized-sign stability proxy), then apply:

1. **Validity filtering:** drop invalid or near-constant factors.
2. **Correlation deduplication:** greedily remove factors that are highly correlated on binarized sequences.
3. **Diversity constraint:** prevent collapse into a single style by enforcing diversity against a fixed anchor proxy.

This is a pragmatic compromise: it reduces the search space while keeping the pool refresh deterministic and easy to audit.

Paper snapshot parameters (deterministic defaults). The offline factor-pool refresher uses: months list $\{12, 24, 48\}$, minimum valid samples per window 200, binarized sparsity threshold $\text{pos_min} = 0.02$ (mark near-constant if $\leq \text{pos_min}$ or $\geq 1 - \text{pos_min}$), greedy binarized-correlation dedup threshold 0.95, target active pool size 12 (bounded to 8–12), and a diversity safeguard that keeps at least 3 factors with $|\text{corr}(\text{factor}_{\text{bin}}, \text{anchor}_{\text{core}})| \leq 0.35$. These parameters are fixed for the reported tables; changing them alters the candidate factor whitelist and therefore the Stage I discrete search space.

Appendix A.4. Stage II Gates (What Must Hold Beyond the Training Objective)

Stage II enforces two robustness gates:

- **Cost stress:** reevaluate shortlisted candidates under perturbed cost parameters and penalize degradation and churn (e.g., switch density and cost/bar).
- **Walk-forward evaluation (WFE):** evaluate a fixed configuration on a sequence of time-shifted windows without refitting, and penalize large drops in worst-window performance.

These gates are intentionally *out-of-objective* relative to the Stage I training-window score, so that selection is driven by stability rather than best-of- N luck. The specific stress grid and WFE thresholds used in this paper snapshot are listed in Table 5.

Setting	min m	MaxDD	cost/bar (bps)	Accept / p05 ≥ 0 (%)
Relaxed	0.02	0.20	4.0	43.5 / 45.8
Default (paper)	0.04	0.15	3.0	18.8 / 68.6
Strict	0.05	0.12	2.0	4.8 / 53.8

Table A.19: **WFE gate threshold sensitivity (4h audit summaries)**. All candidates share the same windowing (win_bars=2000, step_bars=500). We report how different plausible governance thresholds trade off selectivity (accept rate) against tail robustness (fraction of candidates with perturb-holdout p05 ≥ 0 among accepted; p05 denotes the 5th percentile).

Threshold rationale (governance-first).. The numeric thresholds are chosen as conservative governance checks, not tuned to maximize returns. We require a positive worst-window margin (min $m \geq 0.04$) to avoid selecting candidates that fail in at least one window, and we bound mean drawdown and mean cost/bar to avoid high-churn configurations whose apparent profitability depends on unrealistic frictions. As a batch diagnostic, these thresholds accept 18.8% of audited 4h candidates and correlate with more favorable perturb-holdout tails (Table 15), indicating reduced fragility at the cost of selectivity. Table A.19 reports a small sensitivity snapshot: relaxing thresholds increases acceptance but weakens tail robustness, while overly strict thresholds can become too selective (low acceptance) without proportionate tail gains.

Appendix A.5. Walk-Forward Windows (Construction)

Walk-forward evaluation is designed to make time-shift degradation visible without granting the model the ability to refit on each window. Given a long historical span, we partition it into a sequence of overlapping windows (e.g., monthly or multi-week blocks) and evaluate a *fixed* configuration on each window under the same STRICT T+1 semantics and ledger accounting. We summarize WFE by reporting both average performance and worst-window performance (e.g., minimum window-level monthly geometric return proxy), together with churn and cost/bar proxies. This makes it harder for a single lucky window to dominate selection.

Paper snapshot windowing.. For the 4h experiments, we use win_bars= 2000 and step_bars= 500, producing 14 overlapping windows (75% overlap) for the BTC snapshot in Table 14. All candidates compared within a table share the same windowing scheme.

Evidence provenance (paper tables).. Tables 14 and 15 are derived from aggregated audit summaries (per-trial WFE aggregates and perturb-holdout quantiles) rather than requiring the submission package to include large per-window logs. The goal is to keep the decision rules checkable while keeping the compile package minimal.

Appendix A.6. Metric Definitions (Compact)

We report a small set of metrics chosen to balance interpretability and robustness. Let r_t^{net} be net returns from the per-bar ledger and let $R_T = \prod_{t=1}^T (1 + r_t^{\text{net}}) - 1$ be the compounded return.

- **Annualized return (CAGR):** $\text{CAGR} = (1 + R_T)^{\alpha/T} - 1$, where α is bars per year (e.g., 24×365 for 1h).
- **Max drawdown (MaxDD):** $\max_t \left(1 - \frac{E_t}{\max_{u \leq t} E_u}\right)$ where E_t is the equity curve.
- **Turnover and churn:** we proxy churn by switch density (position switches per bar) and by cost/bar (bps) derived from the turnover-proportional ledger.

Window-level reporting applies the same definitions on each window, and we summarize by mean/min across windows.

Appendix A.7. Configuration Knobs

The Stage I search space includes both continuous hyperparameters and discrete module toggles. We keep the description here intentionally high-level (the exact sampled configuration is logged per trial as an audit artifact):

- **Signal horizons:** trend and mean-reversion lookbacks, smoothing, and normalization.
- **Risk overlays:** exposure caps, quantization, cooldown/min-hold rules to suppress churn.
- **Factor channel:** discrete factor choices restricted to the active whitelist, and gating/tilting thresholds.
- **Selection gates:** stress cost multipliers and WFE summary thresholds used in Stage II.

Compute budget (reported snapshot).. Stage I budgets in Table 11 range up to $N = 120$ trials per window. Stage II-A stress evaluates each shortlisted candidate across 9 cost settings in the reported BTC grid snapshot (Table 13). Stage II-B WFE evaluates each candidate across 14 overlapping windows in the reported BTC snapshot (Table 14).

Reproducibility statement

All tables and figures required to compile the manuscript are included in the submission package. The broader codebase implementing the evaluator and selection protocol is maintained in the project repository for traceability and offline regeneration of evidence.

Declaration of competing interests

The author(s) declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

This study uses publicly available market data (open-high-low-close-volume (OHLCV) bars) and perpetual-futures funding rates obtained from exchange endpoints or data providers. Due to exchange terms and licensing, we do not redistribute raw historical data in the submission package. All tables and figures used in the manuscript are included in the submission package (**tables** and **figures**). The broader codebase implementing the evaluator and selection protocol is maintained in the same repository used to prepare this submission.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the author(s) used OpenAI Codex CLI (a generative AI-assisted coding and editing tool) in order to revise and polish the manuscript text, edit $\text{\LaTeX}$ sources, and generate minimal offline scripts for reproducible tables. After using this tool/service, the author(s) reviewed and edited the content as needed and take full responsibility for the content.

References

- [1] D. H. Bailey, J. M. Borwein, M. López de Prado, Q. J. Zhu, The probability of backtest overfitting, *The Journal of Computational Finance* 20 (4) (2017) 39–69. doi:10.21314/jcf.2016.322.
URL <https://doi.org/10.21314/jcf.2016.322>
- [2] R. Sullivan, A. Timmermann, H. White, Data-snooping, technical trading rule performance, and the bootstrap, *The Journal of Finance* 54 (5) (1999) 1647–1691. doi:10.1111/0022-1082.00163.
- [3] H. White, A reality check for data snooping, *Econometrica* 68 (5) (2000) 1097–1126. doi:10.1111/1468-0262.00152.
- [4] P. R. Hansen, A test for superior predictive ability, *Journal of Business & Economic Statistics* 23 (4) (2005) 365–380. doi:10.1198/073500105000000063.
- [5] U. J. Jermann, Pricing perpetual futures, *SSRN Electronic Journal* (2023). doi:10.2139/ssrn.4573912.
- [6] I. Makarov, A. Schoar, Trading and arbitrage in cryptocurrency markets, *Journal of Financial Economics* 135 (2) (2020) 293–319. doi:10.1016/j.jfineco.2019.07.001.
URL <https://doi.org/10.1016/j.jfineco.2019.07.001>
- [7] R. J. Brachman, H. J. Levesque, *Knowledge Representation and Reasoning*, Morgan Kaufmann, 2004.
- [8] B. G. Buchanan, E. H. Shortliffe, *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*, Addison-Wesley, 1984.
- [9] M. López de Prado, *Advances in Financial Machine Learning*, John Wiley & Sons, 2018.
- [10] T. Akiba, S. Sano, T. Yanase, T. Ohta, M. Koyama, Optuna: A next-generation hyperparameter optimization framework, in: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631. doi:10.1145/3292500.3330701.
URL <https://doi.org/10.1145/3292500.3330701>

- [11] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *Journal of Machine Learning Research* 13 (2012) 281–305.
URL <https://jmlr.org/papers/v13/bergstra12a.html>
- [12] M. López de Prado, Seven sins of quantitative investing, *The Journal of Portfolio Management* 45 (1) (2019) 12–25.
- [13] Y. Liu, A. Tsyvinski, Risks and returns of cryptocurrency, *Review of Financial Studies* 34 (6) (2020) 2689–2727. doi:10.1093/rfs/hhaa113.
- [14] Y. Liu, A. Tsyvinski, X. Wu, Common risk factors in cryptocurrency, *The Journal of Finance* 77 (2) (2022) 1133–1177. doi:10.1111/jofi.13119.
- [15] D. Ackerer, J. Hugonnier, U. Jermann, Perpetual futures pricing, Tech. Rep. w32936, National Bureau of Economic Research (2024). doi:10.3386/w32936.
- [16] J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, in: *Advances in Neural Information Processing Systems*, Vol. 24, 2011, pp. 2546–2554.
URL https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html
- [17] F. Hutter, L. Kotthoff, J. Vanschoren, *Automated Machine Learning: Methods, Systems, Challenges*, Springer, Cham, 2019. doi:10.1007/978-3-030-05318-5.
URL <https://doi.org/10.1007/978-3-030-05318-5>
- [18] J. Snoek, H. Larochelle, R. P. Adams, Practical bayesian optimization of machine learning algorithms, in: F. Pereira, C. J. C. Burges, L. Bottou, K. Q. Weinberger (Eds.), *Advances in Neural Information Processing Systems*, Vol. 25, Curran Associates, Inc., 2012, pp. 2951–2959.
URL https://papers.nips.cc/paper_files/paper/2012/hash/05311655a15b75fab86956663e1819cd-Abstract.html